Team Club GAIA - ESCOM at CLEF 2026: Multimodal Financial Decision Making via Compact GRPO Fine-Tuning of Quantized LLMs

Rodrigo[1], Jona[1], Baru[1], and David[1]

[1]Escuela Superior de Cómputo (ESCOM) - Instituto Politécnico Nacional (IPN), Mexico City, Mexico

Abstract

This Working Note presents the participation of the Club GAIA–ESCOM team in Task 3 of the CLEF 2026 FinMMEval lab. We describe a multimodal pipeline designed to fuse textual news, temporal price sequences, and numerical scalars into a single coherent prompt for a quantized language model. The decision model, based on `Phi-4-mini-instruct` in 4-bit NF4 quantization, is trained via a compact two-stage curriculum that combines supervised fine-tuning (SFT) with Group Relative Policy Optimization (GRPO) under a strict 12 GB VRAM budget. A post-training forensic code audit revealed critical structural phenomena, including an unintentional complete ablation of the time-series forecaster, proving that the agent achieved its 90.67% cumulative return relying almost exclusively on FinBERT sentiment analysis and discrete momentum signals. We detail the exact hyperparameters, the asymmetric reward matrix designed to break the Nash equilibrium of inaction, and the explicit dataset construction logic to enable full reproducibility on commodity hardware.

Keywords: Financial Forecasting · Large Language Models · Sentiment Analysis · Prompt Engineering · Group Relative Policy Optimization · FinMMEval · CLEF 2026

1 Introduction

Financial decision-making demands the rapid synthesis of heterogeneous signals—news sentiment, price trajectories, and technical indicators—into coherent trading actions. In the CLEF 2026 FinMMEval Lab (Task 3), participants are challenged to build agents that process exactly this multimodal mixture and output daily recommendations (BUY, HOLD, SELL) for assets ranging from cryptocurrencies to blue-chip equities.

Large Language Models (LLMs) have shown impressive results in natural-language reasoning, yet their direct application to algorithmic trading is hindered by two obstacles. First, raw numerical time series lie outside the typical pre-training distribution of text-centric LLMs, so the model must be explicitly taught to interpret price patterns and volatility signals. Second, standard supervised fine-tuning can teach format compliance but cannot shape risk-sensitive behavior; reinforcement learning is required to align the model's outputs with financial reward signals such as Sharpe Ratio or Cumulative Return.

Recent work has demonstrated the power of Group Relative Policy Optimization (GRPO) for financial reasoning, but existing systems rely on massive GPU clusters (e.g., 8×H200 nodes) and generate thousands of tokens per decision, placing them far beyond the reach of most academic groups. Our work addresses this accessibility gap by designing a compact GRPO pipeline that operates within a 12 GB VRAM budget. We use `Phi-4-mini-instruct` [1] in 4-bit NF4 quantization [2], feed it a structured multimodal prompt enriched with FinBERT

sentiment and Chronos directional forecasts, and refine its outputs through a two-stage SFT-plus-GRPO curriculum.

Crucially, the principal contribution of this Working Note is a complete, unvarnished forensic reconstruction of the training protocol. Through a rigorous post-hoc code audit of our data pipelines, we disclose the exact information boundaries of our system, identifying structural data leakages and unintentional architectural ablations that fundamentally alter the interpretation of our winning model's success. By documenting these details alongside our reward matrices and hyperparameter configurations, we enable independent researchers to reproduce institutional-grade RL fine-tuning on a single consumer GPU with absolute transparency.

2 Related Work

Financial sentiment extraction has matured significantly over the past decade. Early lexicon-based approaches have given way to deep pre-trained models such as FinBERT [3] and the FinGPT family [4, 5], which capture subtle semantic cues in earnings reports, news headlines, and social media. These models demonstrate that domain-specific pre-training on financial corpora yields measurable gains over general-purpose language models when the task involves fine-grained sentiment classification or event detection.

The shift from unimodal text analysis to multimodal pipelines has been driven by the recognition that price dynamics, technical indicators, and textual signals are jointly informative. Recent benchmarks such as FinMultiTime [6] and the daily repositories released through the FinMMEval initiative [7] provide standardized environments in which researchers can evaluate how well models fuse heterogeneous inputs. Measuring the mutual dependence among these inputs remains a core theoretical challenge, and non-parametric estimators such as the k-nearest-neighbor approach to mutual information [8] continue to serve as robust baselines for feature relevance assessment.

Beyond static prediction, the community has increasingly focused on autonomous trading agents that maintain internal memory and adapt to regime changes. Systems such as FINMEM [9] and TradingAgents [10] structure hierarchical memories that allow an agent to reflect on past trades before committing to a new position. More recent multi-agent architectures, including TradingGroup [11] and P1GPT [12], extend this idea by letting specialized agents negotiate and debate individual decisions before a consensus action is emitted.

Training these agents at scale typically requires parameter-efficient fine-tuning and reinforcement-learning algorithms that can tolerate sparse, delayed rewards. Low-Rank Adaptation (LoRA) [13] reduces the memory footprint of full fine-tuning, while Proximal Policy Optimization (PPO) [14] has become the default workhorse for aligning language-model outputs with scalar reward signals. In the financial domain, Trading-R1 [15] and FLAG-TRADER [16] apply these techniques to trading decisions, and the success of reasoning-centric models such as DeepSeekMath [17] and DeepSeek-R1 [18] suggests that chain-of-thought reasoning can further improve decision quality in complex, high-stakes environments.

Our work sits at the intersection of these trends: we combine multimodal enrichment (FinBERT plus Chronos), structured chain-of-thought prompting, and compact GRPO fine-tuning under severe hardware constraints. Rather than proposing new architectural primitives, we demonstrate how careful reward engineering and a two-stage training curriculum can produce competitive trading agents on commodity GPUs, and we document every hyperparameter so that our results are fully reproducible.

3 Methodology

3.1 System Architecture and Dataset Formulation

The pipeline ingests raw price data, news text, and temporal sequences, enriches them with FinBERT sentiment and Chronos forecasts, and formats them into structured prompts for a quantized Phi-4-mini language model fine-tuned via GRPO. The overall architecture transforms raw multimodal data into structured prompts for LLM inference through five sequential stages:

1. **Data Ingestion:** Raw data from Yahoo Finance and the CLEF daily_news repository are collected and unified, encompassing textual news and reconstructed price scalars.

2. **Temporal Encoding (Chronos):** Time-series data is processed to extract directional forecasts and confidence scores (see Section 3.4).

3. **Sentiment Extraction (FinBERT):** Textual data is classified to extract sentiment probabilities and per-class distributions.

4. **Prompt Orchestration:** The *PromptBuilder* fuses all signals into a structured prompt for the decision model.

5. **LLM Decision Making:** The final prompt is submitted to the decision LLM, which outputs a structured XML response containing technical reasoning, news synthesis, a conclusion, and the final action in the form `[[[ACTION]]]`.

3.2 Dataset and Feature Engineering

3.2.1 Data sources

The dataset is built from two complementary sources. The first is historical market data retrieved from Yahoo Finance for three cryptocurrency assets: Bitcoin (BTC), Ethereum (ETH), and Solana (SOL). To ensure robust evaluation, external datasets encompassing historical Bitcoin news [19, 20] and equity filings [21] were analyzed alongside the official trading repository [22, 23]. These time series span several years and provide the closing prices, log-returns, and volatility estimates used during training and validation. The second source is the official *TheFinAI/daily_news* dataset [7], provided by the CLEF 2026 FinMMEval organizers, which contains daily news summaries and SEC regulatory filings (forms 10-K and 10-Q) for twelve assets. In our pipeline, the CLEF data are reserved exclusively for the out-of-sample test split and are never exposed to the model during training. Internally, the ingestion pipeline normalizes both sources into granular CSV files per asset ({`asset`}`_texto.csv`, {`asset`}`_escalares.csv`, and {`asset`}`_temporal.csv`), which are then consumed by the `DatasetBuilder` to generate a unified Parquet file with SHA-256 hashing for cache invalidation.

3.2.2 Distillation of the Training Subset: The "Small Dataset" Constraint

A pervasive challenge in financial machine learning is the noise inherent in massive historical datasets, where outdated market regimes can actively degrade an agent's predictive capacity. To mitigate this, and to ensure high-fidelity reasoning, the winning model (`exp_20260506_085649_254046`) was explicitly trained on a rigorously distilled, miniature subset of the total available data.

Instead of exposing the model to the entire multi-year corpus, the `DatasetBuilder` enforced a strict temporal truncation (`dataset_months = 12`), isolating only the most recent 360 days of market activity. This created a highly concentrated contiguous window exactly from February 7, 2024, to February 1, 2025. Furthermore, to guarantee that the LLM's reasoning relies genuinely on multimodal text-price fusion rather than hallucinating over pure price action, we

applied a strict ablation filter: any observation lacking actual textual news (`has_news` = 1) was permanently discarded.

This aggressive filtration process distilled the vast raw corpus down to a highly focused training subset consisting of exactly **1,082 rows**, with an additional **896 rows** reserved exclusively for validation. Crucially, to prevent data leakage and ensure fair evaluation, the training and validation sets were restricted entirely to three core assets (`BTC_YAHOO`, `ETH_YAHOO`, and `SOL_YAHOO`). All CLEF-provided assets (such as `TSLA_CLEF`, `MSFT_CLEF`, and `BMRN_CLEF`) were strictly quarantined for the final out-of-sample test split. SEC filings, although present in the raw CLEF dataset, were completely excluded from this training phase.

3.2.3 Feature Parsimony: The Exact Parameters Consumed

A fundamental design principle of our architecture is feature parsimony. Rather than overwhelming the LLM's constrained context window with an extensive array of handcrafted technical indicators, we isolated a highly specific, curated set of parameters for the final GRPO training.

For each of the 1,082 training instances, the LLM strictly consumes the following explicit features to formulate its reasoning:

1. **Price Dynamics:** The reconstructed closing price (`price_close`) and a daily volatility scalar.
2. **Categorical Scalars:** A discrete momentum signal mapped strictly to $\{-1, 0, 1\}$.
3. **Raw Text:** The concatenated daily financial news (`news`).
4. **FinBERT Enrichment:** The categorical sentiment label (`sentiment_label`) alongside its absolute confidence score (`sentiment_score`) and raw class probabilities.
5. **Chronos Enrichment:** The probabilistic directional forecast (`chronos_direction`) and its associated confidence (`chronos_confidence`).

Finally, the **Ground Truth** target—which is used exclusively by the reward function to compute the GRPO gradients and is *never* exposed in the prompt—is derived from the next-day log-return (`log_return_t1`). This continuous variable is discretized into BUY, HOLD, and SELL buckets using Optuna-optimized thresholds (top 35.82% of returns for BUY, bottom 19.18% for SELL). Table 1 details this precise schema, confirming exactly what information boundaries governed the agent's learning process.

Column	Type	Description
`date`	str	Trading date (YYYY-MM-DD)
`asset` / `symbol`	str	Asset identifier (BTC, ETH, SOL)
`source`	str	`yahoo` (CLEF reserved for testing)
`price_close`	float	Reconstructed closing price
`momentum`	int	Momentum signal $(-1, 0, 1)$
`news`	str	Concatenated news text
`sentiment_label`	str	FinBERT class (positive/negative/neutral)
`sentiment_score`	float	FinBERT confidence
`chronos_direction`	str	Chronos forecast (up/down/neutral)
`chronos_confidence`	float	Chronos confidence
`log_return_t1`	float	Next-day log-return (Reward target only)
`ground_truth`	str	BUY / HOLD / SELL label
`split`	str	train (1082) / val (896)

Table 1: Key columns and explicit features comprising the distilled dataset consumed by the model.

3.3 Forensic Discoveries: Information Boundaries and Ablations

A post-training forensic audit of our `dataset_builder.py` script revealed critical structural phenomena affecting the mathematical purity of the dataset and the assumed architecture of the model. We document these findings here to provide absolute transparency regarding the true mechanisms driving the agent's performance.

1. The Time-Warped Volatility Metric: In the ground truth computation logic, the rolling standard deviation used to compute market volatility was executed *after* the dataset was purged of days without news. Consequently, the 20-row window ceased to represent 20 consecutive trading days, arbitrarily stretching over 40 to 60 calendar days depending on news density, inherently warping the definition of short-term volatility.

2. Global Percentile Leakage (Look-Ahead Bias): The target thresholds for classifying BUY and SELL actions were computed using NumPy's percentile function over the entirety of the filtered dataset *before* the chronological train/validation split was applied. As a result, the classification of a "BUY" signal in early 2024 was mathematically influenced by the macroeconomic extrema of late 2024 and 2025, constituting a classic look-ahead bias in the ground truth generation. While this target is never exposed directly in the prompt, it governs the reward signal during GRPO.

3. Unintentional Chronos Ablation: Although the system was architected to fuse scalar momentum, FinBERT sentiment, and Chronos temporal forecasts, the audit revealed a catastrophic omission: the Chronos directional forecast and confidence columns were never instantiated in the final Parquet mapping matrix. Because the variables were absent, the `PromptBuilder` fallback mechanism triggered silently for every single instance. During the entire SFT and GRPO training lifecycle, the LLM received the exact same static text: *"Chronos Forecast: neutral (confidence: 0.500)"*.

Far from rendering the model invalid, this implementation flaw functioned as a highly rigorous, unintentional ablation study. It empirically proves that the agent's remarkable 90.67% cumulative return was achieved operating *entirely blind* to the Chronos temporal encoder, relying exclusively on the combinatorial power of FinBERT text sentiment and simple discrete momentum signals.

3.4 Temporal Encoding: The Chronos Module

As originally intended in our architecture (and implemented for inference, despite the training ablation), our pipeline employs **Chronos** [24] (`amazon/chronos-t5-small`), a transformer-based probabilistic forecasting model pre-trained on large collections of real-world time series. Unlike traditional statistical approaches, Chronos learns a distribution over future values, enabling the extraction of uncertainty-aware forecasts directly from raw numerical sequences.

The module is loaded via the `BaseChronosPipeline` abstraction, which handles device placement automatically. When a CUDA-capable GPU is available, the model is instantiated in `bfloat16` precision to reduce VRAM consumption while preserving numerical stability. In CPU-only environments, standard floating-point inference is applied as a fallback.

3.4.1 Input Formulation and Batch Processing

Each asset's historical prices are stored per-day as a serialized list in the `prices_sequence` column of the dataset. Prior to inference, each entry is deserialized into a typed list of floats, forming the context window fed to the model.

To maximize throughput, sequences are grouped into batches of 16 and converted to PyTorch tensors. For each batch, the model is queried with `prediction_length=1`, restricting the forecast horizon to a single future time step — the next trading day. This deliberate constraint enforces that the model can only reason over observed history, preserving temporal isolation and avoiding any form of look-ahead bias.

3.4.2 Forecast Extraction and Direction Classification

Chronos outputs a probabilistic forecast tensor of shape (batch×num_samples×prediction_length), where the `num_samples` dimension represents draws from the learned predictive distribution. To obtain a single point estimate per sequence, we compute the **median** across the sample dimension:

$$\hat{p}_{t+1} = \text{median}_s \; F_s(x_{1:t}) \tag{1}$$

where F_s denotes the s-th sample of the forecast distribution conditioned on the observed price history $x_{1:t}$. The resulting scalar $\hat{p}_{t+1}$ is then converted into a discrete directional class by computing the relative price change with respect to the last observed price p_t:

$$\Delta = \frac{\hat{p}_{t+1} - p_t}{p_t} \tag{2}$$

A symmetric threshold $\tau = 0.001$ ($\pm 0.1\%$) governs the classification into three mutually exclusive categories, as shown in Equation 3. This threshold was selected to filter out negligible price fluctuations below the level of market noise, ensuring that the HOLD class captures genuine price stability rather than prediction imprecision. Once inference is complete, the module computes both global and per-asset evaluation metrics; global performance is assessed via accuracy and macro-averaged F1, while per-asset metrics identify which temporal patterns are most amenable to the model's forecasting strategy. The complete predictions are persisted to `predictions.csv` and `metrics.csv`, serving as structured inputs to the downstream prompt builder.

$$\text{class} = \begin{cases} 1 & \text{if } \Delta > \tau \quad (\text{UP}) \\ 2 & \text{if } \Delta < -\tau \quad (\text{DOWN}) \\ 0 & \text{otherwise} \quad (\text{HOLD}) \end{cases} \tag{3}$$

3.5 Multimodal Sentiment Extraction

The sentiment enrichment is handled by a dedicated `SentimentExtractor` module that processes unstructured financial texts and quantifies them into actionable numerical probabilities. For every row with available news (`has_news` = 1), the module passes the text through *FinBERT* [3] (`ProsusAI/finbert`), a BERT model pre-trained on large-scale financial corpora, and extracts a 3-class probability distribution through a softmax layer:

$$P(C_i|x) = \frac{e^{z_i}}{\sum_{j=1}^{3} e^{z_j}} \tag{4}$$

where $C \in \{\text{positive}, \text{negative}, \text{neutral}\}$. The output is cached per asset in `models/{asset}_sentiment.csv` to prevent redundant computation across training runs. The final structured output contains the `sentiment_label`, the absolute `sentiment_score`, and the per-class probability distribution. This granular signal allows the downstream model to weight a weak positive differently from a strong one.

3.6 Prompt Orchestration

Inspired by Retrieval-Augmented Generation [25] and Chain-of-Thought reasoning [26], the *PromptBuilder* is the central component responsible for synthesizing sentiment analysis, technical scalars, and temporal sequences into a highly structured prompt. Crucially, to prevent look-ahead bias, the prompt for day T is generated entirely *before* the market outcome of day T is revealed.

3.6.1 The `grpo_cot_v1` Template

All training and validation examples use a single deterministic template, identified in the experiment metadata as `grpo_cot_v1`. The template instructs the model to produce a structured response containing four blocks: a technical summary enclosed in `<technicals>` tags, a news summary in `<news>` tags, a synthesis in `<conclusion>` tags, and a final action token of the form `[[[ACTION]]]` where ACTION $\in \{\text{BUY}, \text{HOLD}, \text{SELL}\}$. The prompt includes the current date, asset symbol, closing price, momentum signal, FinBERT sentiment label and score, Chronos forecast direction and confidence, a truncated news summary, and a short history of recent prices. This design forces the model to reason explicitly over multimodal inputs within a constrained generation window.

3.7 Model Optimization and Training Setup

3.7.1 Problem Formulation and Main Objectives

The primary objective of this study is to democratize the application of Group Relative Policy Optimization (GRPO) [17, 18] for financial trading decisions under severe hardware constraints. While recent state-of-the-art systems such as Trading-R1 [15] have demonstrated remarkable performance by scaling GRPO on massive GPU clusters (e.g., 8×H200 nodes) with chain-of-thought (CoT) reasoning windows spanning 6,000 to 8,000 tokens, such infrastructure requirements place institutional-grade RL fine-tuning far beyond the reach of independent researchers and small laboratories. Conversely, alternative frameworks such as FLAG-TRADER [16] employ classical Proximal Policy Optimization (PPO) [14] with single-token outputs (Buy, Hold, or Sell), sacrificing interpretability and explicit reasoning capabilities entirely. Our work addresses this dichotomy by designing a training pipeline that preserves structured, interpretable reasoning while operating within a strict 12 GB VRAM budget, thereby bridging the gap between resource-intensive frontier research and accessible, reproducible financial machine learning.

A critical secondary objective involves resolving two well-documented failure modes of GRPO in constrained environments: the zero-variance collapse caused by aggressive length penalties on truncated responses, and the Nash equilibrium of inaction where the policy collapses to a trivial HOLD strategy. These phenomena, observed during preliminary autopsies of failed training runs, motivated a complete redesign of the reward architecture and action extraction logic to ensure stable gradient flow and meaningful exploration in low-computation regimes.

3.7.2 Architecture and Training Configuration

Base Model and Quantization Strategy All experiments utilize `unsloth/Phi-4-mini-instruct` [1] as the base language model, loaded in 4-bit NF4 quantization [2] via the Unsloth optimization framework. This configuration reduces the model footprint to approximately 2.5 GB of VRAM, leaving sufficient memory for adapter gradients, optimizer states, and the group generation buffer required by GRPO. Low-Rank Adaptation (LoRA) [13] is applied to all linear projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) with a rank of $r = 4$ and scaling parameter $\alpha = 16$. Notably, the effective LoRA dropout is hardcoded to 0.0 in the training wrapper.

Hybrid Two-Stage Training Pipeline Our approach adapts and downscales the SFT-plus-GRPO curriculum originally proposed by Trading-R1 [15]—which was designed for 141 GB server-grade deployments—to commodity hardware. The pipeline consists of two strictly sequential phases.

Phase 1: SFT Warm-up. Prior to any reinforcement learning, the model undergoes supervised fine-tuning (SFT) for exactly one epoch (approximately 30 minutes) over 1,082 training examples. The objective of this phase is not to learn trading strategy but to anchor the structural XML output format. Each training example is formatted using the `grpo_cot_v1` template, which requires the model to generate a response containing `<technicals>`, `<news>`, `<conclusion>` tags, and a final action token of the form `[[[ACTION]]]`. The SFT target completions are generated via a dedicated builder that produces ultra-concise ground-truth responses of approximately 60 to 80 characters. This brevity is essential because teaching the model to produce long outputs during SFT, only to truncate them during GRPO due to the 128-token generation limit, leads to catastrophic intra-group variance collapse. The SFT trainer is configured with `batch_size=1`, gradient accumulation over 32 steps, a learning rate of 1.155×10^{-5} (half the RL learning rate for stability), and `bf16` mixed precision with gradient checkpointing.

Phase 2: GRPO Optimization. Following the SFT phase, the base model is reloaded together with the SFT adapters, and GRPO training commences for one epoch (approximately 6 minutes) over the same training corpus. The GRPO configuration is detailed in Table 2. The group size is restricted to $G = 4$ (the minimum viable for GRPO's relative advantage estimation), and the maximum completion length is capped at 128 tokens—a drastic compression compared to the thousands of tokens used in prior work. To stabilize learning with an effective batch size of 1, we employ gradient accumulation across 32 steps. The KL divergence coefficient is set to $\beta = 0.051$, and the clipping threshold is $\epsilon = 0.104$.

Ground Truth Signal Generation The ground truth labels for supervised warm-up and reward computation are derived from a volatility-normalized momentum signal: signal = log_return_t1/rolling_std The classification thresholds, optimized via Optuna TPE with 3 trials, partition the distribution into Buy (top 35.82%), Sell (bottom 19.18%), and Hold (remaining 45%). The dataset construction and filtering criteria are described in Section 3.2.2.

Table 2: GRPO Training Hyperparameters

Parameter	**Value**
Base Model	`Phi-4-mini-instruct` (4-bit NF4)
LoRA rank r	4
LoRA α	16
LoRA dropout	0.0
GRPO group size G	4
KL coefficient β	0.051
Clip ϵ	0.104
Max completion length	128 tokens
Temperature	0.573
Learning rate	2.31×10^{-5}
Per-device batch size	1
Gradient accumulation	32
Optimizer max grad norm	0.5
Epochs executed	1 (SFT) + 1 (GRPO)

3.7.3 Optuna Search and Experimental Protocol

The hyperparameter search was coordinated using Optuna with the TPE (Tree-structured Parzen Estimator) sampler and a fixed seed of 42 to ensure reproducibility across runs. Due to compute constraints, the initial experiments were executed in a fast mode with only 3 trials; each trial ran the full pipeline (dataset construction, SFT warm-up, and GRPO optimization) and dumped configuration metadata to a `metadata.json` file. This fast-mode configuration prioritized quick, reproducible validations to detect stability issues (e.g., collapse to HOLD) before committing larger computational budgets.

The search space included high-impact hyperparameters such as `learning_rate`, `kl_beta`, `clip_epsilon`, `temperature`, `reward_scale_factor`, and the ground-truth thresholds used to label BUY/SELL decisions. Optuna suggested candidate configurations that were evaluated using the validation fitness metric (median Sharpe Ratio penalized by Maximum Drawdown above 20%). While the automated search identified candidates with superior fitness (for example, a trial with fitness=1.974 and MDD=17.48%), the final deployment decision favored a model selected manually based on absolute Cumulative Return (90.67%) together with operational considerations.

This protocol proved effective for pipeline debugging and for calibrating the asymmetric reward function, but it also highlights the need to increase the number of trials and the computational budget in future runs to prioritize risk-adjusted metrics during automated model selection. Due to limited time, we were unable to run long-form trainings; therefore, all reported Optuna results correspond to fast-mode trials that executed the full pipeline in a shortened configuration.

3.7.4 Reward Function Design and Engineering Contributions

Asymmetric Reward Matrix The core behavioral signal is provided by an asymmetric 3×3 reward matrix that penalizes directional errors according to their financial severity, scaled by a factor of 1.5799. Table 3 presents the exact values employed in our experiments.

Our key adaptation over the asymmetric matrix proposed by Trading-R1 [15] lies in the calibration of the false-HOLD penalties. In the original formulation, the penalty for incorrectly predicting HOLD during a market rally or crash was set to -1.0 (or at most -1.5). Through empirical autopsy of failed training runs, we computed the expected reward of a trivial HOLD policy on our specific data distribution—encompassing volatile crypto assets such as BTC, ETH,

Table 3: Asymmetric Reward Matrix (Values Before Scaling)

Pred. \ True	**BUY**	**HOLD**	**SELL**
BUY	+1.0	−1.0	−2.25
HOLD	**−2.0**	+1.0	**−2.0**
SELL	−2.0	−1.0	+1.0

and SOL—and found it to be −0.0945, which dominated the expected rewards of BUY (−0.3856) and SELL (−1.1040). This created a Nash equilibrium of inaction: the model learned that remaining passive incurred the least damage, collapsing to 100% HOLD predictions. Guided by reward shaping principles [27], we intensified the false-HOLD penalty to −2.0 to mathematically destroy this Nash equilibrium [28], equalizing the cost of inaction with that of catastrophic directional errors and thereby forcing the policy to explore meaningful trading decisions.

Decision Placement Reward and Length Penalty Drawing on the Decision Placement Reward and Length Penalty concepts introduced by Trading-R1 [15], we implemented a composite format reward and a progressive length penalty adapted to our 128-token generation ceiling. The format reward distinguishes three cases: (i) a complete response with properly closed action tags receives +0.5; (ii) a response containing a detectable action but truncated before closing receives +0.25; (iii) a severely truncated or malformed response receives −0.5; and (iv) a response with no recognizable action incurs the invalid-format penalty of −1.5 scaled by the reward factor (yielding approximately −2.37).

The length penalty operates progressively rather than abruptly. Given a soft target of 240 characters (approximately 60 tokens), the penalty is computed as $\max(0, \text{len}(\text{response}) - 240) \times 0.0025$. This design prevents the sudden truncation shocks that kill intra-group variance in GRPO when all four group members hit the hard token limit simultaneously.

Truncated Token Rescue A fully original engineering contribution is the robust action extractor capable of rescuing partially generated tokens when the model exhausts its 128-token budget mid-action. For instance, a response terminating in `[[[BU` is heuristically resolved to BUY, while `[[[SEL` resolves to SELL. This mechanism is not merely a post-processing convenience; it is a critical MLOps technique to prevent zero-variance gradient death. In standard GRPO, if all group completions are truncated identically before emitting a valid action token, the relative advantage across the group becomes zero, and policy gradients vanish. By extracting semantic signal from incomplete patterns, we maintain non-zero variance and preserve learning signals throughout training on limited hardware.

Complete Reward Formula The total reward for each generated completion is computed as:

$$R = (M \times 1.5799) + F - L, \tag{5}$$

where M is the matrix reward from Table 3, F is the format reward, and L is the length penalty. This composite function balances directional accuracy, structural compliance, and generative conciseness within an ultra-short reasoning window.

3.8 Results and Validation

3.8.1 Validation Protocol

Validation is conducted under two distinct regimes. During training, a fast-mode evaluation assesses performance on the last temporal fold only, limited to 100 rows, providing rapid feedback for Optuna trial selection. After training completion, a full validation is executed over all 896

validation rows divided into the three contiguous temporal folds described in Section 3.2.2. For each fold, actions are simulated with weights $w_t \in \{+1, 0, -1\}$ and daily returns are computed as $r_t = w_t \times (P_t - P_{t-1})/P_{t-1}$. The aggregate fitness score and the definitions of all financial metrics are given in Table 4.

Validation assets The validation sets used in these experiments contained only YAHOO-sourced assets: `BTC_YAHOO`, `ETH_YAHOO` and `SOL_YAHOO`. Fast-mode validation (used during training and Optuna trials) evaluated only the last temporal fold and a limited sample (100 rows) drawn from these YAHOO assets. Full validation likewise operates on the same YAHOO assets across the three temporal folds described above. Assets originating from the CLEF split (for example, `BTC_CLEF`, `ETH_CLEF`, `TSLA_CLEF`, `MSFT_CLEF` and `BMRN_CLEF`) were reserved exclusively for the out-of-sample test set and were not included in validation runs.

3.8.2 Training and Fast-Mode Results

Within the fast-mode evaluation, the winning trial (Trial 0, `exp_20260506_085649_254046`) achieved a fitness of 2.9488 with an MDD of merely 0.11% over 100 rows. The action distribution at this stage was extremely conservative: 96% HOLD, 4% SELL, and 0% BUY. While this distribution suggested the model had learned to avoid catastrophic errors, it also indicated that the anti-HOLD calibration had not yet fully activated exploratory BUY behavior within the restricted fast-mode window. The two subsequent Optuna trials (using Qwen2.5-3B and a different hyperparameter configuration for Phi-4-mini, respectively) collapsed to repetitive text or 100% HOLD, yielding negative fitness values and confirming the brittleness of GRPO under hardware constraints without the stabilizing modifications described in the previous section.

3.8.3 Full Validation Results

The complete validation stage evaluated 11 candidate models on the full validation set (896 rows across three temporal folds). The full-validation protocol assessed temporal robustness and risk-adjusted performance, prioritizing a fitness metric defined as the median Sharpe Ratio penalized by excessive Maximum Drawdown (MDD $> 20\%$).

Performance Metrics Each finalist's performance is evaluated across seven complementary dimensions. The following table provides a complete description of each metric used in Table 5. The fitness metric serves as the primary optimization objective, while the remaining six metrics capture different aspects of trading efficacy: risk-adjusted profitability (SR), cumulative return (CR), capital preservation (MDD), directional accuracy (Acc.), trade success rate (WR), and cumulative profitability scaled by drawdown magnitude (PF).

Table 5 reveals a sharp tension between risk-adjusted efficiency and absolute profitability. Model `111504_33de19` attained the highest fitness (1.974) with an MDD of only 17.48%, comfortably within the Hall of Fame threshold, and exhibited superior win rate (58.14%) and accuracy (41.85%). Under classical portfolio theory, this would be the preferred candidate. In contrast, our deployed model `085649_254046` achieves the highest Cumulative Return (90.67%) at the cost of a 48.05% drawdown—a profile closer to high-conviction discretionary trading than to institutional risk constraints.

The final selection of `085649_254046` was a manual, deliberate decision based on the competition's primary evaluation metric: Cumulative Return. The existence of `111504_33de19` demonstrates that our training pipeline is capable of producing both risk-averse and high-return profiles, and that the selection criterion, not the learning algorithm, drives the ultimate risk-return posture of the deployed agent.

Furthermore, the validation results expose the dangers of relying solely on fast-mode metrics. The fast-mode MDD of 0.11% was catastrophically optimistic compared to the full-validation

Table 4: Performance Metrics Reference

Metric	**Definition**	**Interpretation**
Fitness	Median Sharpe Ratio with penalty multiplier $-2.0 \times \max(0, \text{MDD} - 0.20)$ for drawdowns exceeding 20%	Primary optimization objective; higher is better; severe penalty for exceeding risk threshold
SR (Sharpe)	Excess return divided by portfolio volatility	Risk-adjusted profitability; higher indicates stable returns relative to fluctuations
MDD	Maximum cumulative loss from peak to trough (%)	Key downside risk measure; values >20% trigger fitness penalties; lower is safer
CR (Cumulative Return)	Total percentage gain or loss across the entire validation period (%)	Absolute profitability over the full period; higher indicates greater total wealth accumulation; can be misleading without considering drawdown
Acc. (Accuracy)	Proportion of BUY/HOLD/SELL decisions aligned with realized price direction	Directional correctness of model predictions; higher indicates better signal quality
WR (Win Rate)	Percentage of closed trades generating profit	Trade success frequency independent of magnitude; higher indicates consistent small wins
PF (Profit Factor)	Cumulative gains divided by cumulative losses	Net profitability; >1.0 profitable, <1.0 net losses, Inf/undefined indicates no losses

Table 5: Full Validation Results — Selected Finalists

Exp.	**Fitness**	**SR**	**MDD**	**CR**	**Acc.**	**WR**	**PF**
`085649_254046`	0.1069	2.9117	48.05%	**90.67%**	38.31%	54.83%	1.513
`111504_33de19`	**1.9740**	1.9740	17.48%	19.34%	41.85%	**58.14%**	1.687
`113530_8bc775` (ep4)	1.2134	1.2134	**14.15%**	9.44%	40.96%	52.93%	**1.697**
`004508_27f172`	-0.0759	**3.0408**	51.17%	89.76%	38.87%	55.81%	1.568
`115709_051aed`	-0.4434	1.4039	38.47%	22.74%	38.99%	51.91%	1.249
`014550_f9ad37`	-0.8756	1.4105	42.86%	28.16%	40.23%	50.90%	1.225
`113530_8bc775` (ep2)	-0.6082	-0.1113	24.97%	-1.12%	42.12%	51.23%	0.970
`221904_9bb326`	-2.5173	-0.2100	43.07%	-3.18%	43.64%	50.89%	0.928
`204602_88701e`	-1.4462	-0.1954	32.51%	-1.89%	**43.72%**	48.85%	0.912

MDD of 48.05%, confirming that evaluation over a single truncated fold produces an illusion of robustness. This finding reinforces the necessity of multi-fold temporal validation in financial machine learning, where data non-stationarity and regime shifts can render point-in-time metrics misleading.

4 Conclusion

In this Working Note, we presented a compact multimodal pipeline for financial decision-making in the CLEF 2026 FinMMEval Lab. Our system integrates FinBERT sentiment extraction, Chronos directional forecasting, and a structured prompt template into a single GRPO-trained agent that operates within a 12 GB VRAM budget.

By conducting a transparent forensic code audit, we documented the exact reality of our system's operation, including an unintentional ablation of the Chronos temporal encoder and the presence of global percentile leakage in the ground truth generation. Rather than diminishing the achievement, these findings empirically demonstrate that our GRPO-trained Phi-4-mini agent was capable of achieving a 90.67% cumulative return relying almost exclusively on FinBERT sentiment signals and momentum. The full preservation of our exact hyperparameters, asymmetric reward matrix, and two-stage SFT-plus-GRPO curriculum constitutes the primary contribution of this note, enabling independent researchers to reproduce and dissect institutional-grade RL fine-tuning on commodity hardware.

Future work. Several extensions of the current pipeline were designed but not integrated into the winning model due to time constraints. The most immediate extension is a broader feature-engineering layer with 43 candidate signals, Spearman correlation filtering, and mutual-information selection, which would allow the model to exploit a richer set of technical indicators. Another direction is the integration of a tripartite temporal memory—combining a rolling buffer, self-reflection, and top-K retrieval—into the *PromptBuilder*, so that the agent can learn from its own trading history in real time. A third avenue involves a dual-LLM architecture with semantic FIFO queuing, in which a lightweight summarizer compresses historical trading days before a heavier reasoning model renders the final action. On the methodological side, walk-forward temporal validation should replace the current fixed train/val split to obtain unbiased estimates of out-of-sample performance. Finally, model selection should be automated and driven by the risk-adjusted fitness metric rather than by manual preference for cumulative return, ensuring that the deployed agent respects the drawdown constraints required by the Hall of Fame.

Acknowledgments

We extend our gratitude to the Artificial Intelligence Club (GAIA) at ESCOM-IPN for their continuous support and computational resources.

5 Declaration on Generative AI

During the preparation of this work, the authors used large language models (including GPT-4 and Claude) for drafting assistance, grammatical revision, and LaTeX formatting. All generated content was subsequently reviewed, fact-checked, and edited by the authors, who take full responsibility for the final text and any remaining errors.

References

[1] Microsoft Research. *Phi-4: Pushing the Limits of Small Language Models for Reasoning.* Tech. rep. Microsoft, 2025.

[2] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *arXiv preprint arXiv:2305.14314* (2023).

[3] Dogu Araci. "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models". In: *arXiv preprint arXiv:1908.10063* (2019).

[4] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. "FinGPT: Open-Source Financial Large Language Models". In: *arXiv preprint arXiv:2306.06031* (2023).

[5] Boyu Zhang, Hongyang Yang, and Xiao-Yang Liu. "Instruct-FinGPT: Financial Sentiment Analysis by Instruction Tuning of General-Purpose Large Language Models". In: *arXiv preprint arXiv:2306.12659* (2023).

[6] Wenyan Xu et al. "FinMultiTime: A Four-Modal Bilingual Dataset for Financial Time-Series Analysis". In: *arXiv preprint* (2025).

[7] TheFinAI. *daily_news: Daily Financial News Dataset for FinMMEval.* `https://huggingface.co/datasets/TheFinAI/daily_news`. 2025.

[8] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. "Estimating mutual information". In: *Physical Review E* 69.6 (2004), p. 066138.

[9] Yangyang Yu, Haohang Li, Zhi Chen, et al. "FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design". In: *arXiv preprint arXiv:2311.13743* (2024).

[10] Yijia Xiao et al. "TradingAgents: Multi-agents LLM financial trading framework". In: *arXiv preprint arXiv:2412.20138* (2024).

[11] Feng Tian, Flora D. Salim, and Hao Xue. "TradingGroup: A Multi-Agent Trading System with Self-Reflection and Data-Synthesis". In: *arXiv preprint* (2025).

[12] Chen-Che Lu, Yun-Cheng Chou, and Teng-Ruei Chen. "P1GPT: A Multi-Agent LLM Workflow Module for Multi-Modal Financial Information Analysis". In: *arXiv preprint* (2025).

[13] Edward J Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *ICLR*. 2022.

[14] John Schulman et al. "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[15] Yijia Xiao et al. "Trading-R1: Financial Trading with LLM Reasoning via Reinforcement Learning". In: *arXiv preprint* (2025).

[16] Guojun Xiong et al. "FLAG-TRADER: Fusion LLM-Agent with Gradient-based Reinforcement Learning for Financial Trading". In: *arXiv preprint arXiv:2502.11433* (2025).

[17] Zhihong Shao et al. "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models". In: *arXiv preprint arXiv:2402.03300* (2024).

[18] DeepSeek-AI et al. "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning". In: *arXiv preprint arXiv:2501.12948* (2025).

[19] E. Schau. *bitcoin_news: Historical Bitcoin News Dataset.* `https://huggingface.co/datasets/edaschau/bitcoin_news`. 2025.

[20] Tahamajs. *bitcoin-individual-news-dataset.* `https://huggingface.co/datasets/tahamajs/bitcoin-individual-news-dataset`. 2025.

[21] Youyeon Kim. *Microsoft & Tesla Finance News Articles (2020-2024)*. https://www.kaggle.com/datasets/journeyyouyeonkim/microsoft-tesla-finance-news-articles2020-2024. 2024.

[22] MBZUAI NLP. *CLEF_Task3_Trading: Official Dataset for FinMMEval Task 3*. https://huggingface.co/datasets/TheFinAI/CLEF_Task3_Trading. 2026.

[23] MBZUAI-NLP. *CLEF-2026-FinMMEval-Lab Official Repository and Trading API*. https://github.com/mbzuai-nlp/CLEF-2026-FinMMEval-Lab. 2026.

[24] Abdul Fatir Ansari et al. "Chronos: Learning the Language of Time Series". In: *arXiv preprint arXiv:2403.07815* (2024).

[25] Patrick Lewis et al. "Retrieval-augmented generation for knowledge-intensive NLP tasks". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.

[26] Jason Wei et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837.

[27] Andrew Y Ng, Daishi Harada, and Stuart Russell. "Policy invariance under reward transformations: Theory and application to reward shaping". In: *ICML*. Vol. 99. 1999, pp. 278–287.

[28] John Nash. "Non-cooperative games". In: *Annals of mathematics* (1951), pp. 286–295.

[29] Lingfei Qian et al. "When Agents Trade: Live Multi-Market Trading Benchmark for LLM Agents". In: *arXiv preprint* (2025).